



Proyecto T2G — Knowledge Graph a partir de Documentos

T2G es una *pipeline modular y extensible* que convierte documentos heterogéneos (PDF, DOCX, imágenes) en una **Representación Intermedia (IR) homogénea**, los enriquece con **contexto semántico global y local**, esto con el fin de poder crear schemas y su selección apoyado del contexto para facilitar y prepara la base para construir **grafos de conocimiento**.

- **Entrada:** PDF / DOCX / PNG / JPG
- **Salidas:**
 - DocumentIR (JSON)
 - DocumentIR+Topics (JSON)
- **Salidas futuras:** Chunks, Mentions, Entities, Triples, Normalización, Grafo.
- **Diseño:** subsistemas **desacoplados**, contratos **Pydantic**, orquestación vía **CLI + YAML**.



Objetivos

- Unificar la ingesta de documentos en una **IR JSON común** independientemente del formato.
- Enriquecer documentos con **contexto semántico a nivel documento y chunk**.
- Mantener una arquitectura **resiliente, escalable y modular**: cada subsistema puede ejecutarse de forma independiente.
- Preparar la base para **grafos de conocimiento, QA empresarial, compliance regulatorio y sistemas RAG**.



Subsistemas

Nº	Subsistema	Rol principal	Entrada	Salida	Estado
1	Parser	Genera IR JSON homogénea con metadatos y layout	Doc (PDF/DOCX/IMG)	DocumentIR JSON	✓
2	Hybrid Contextizer (doc)	Asigna tópicos y keywords globales a nivel documento	DocumentIR	DocumentIR+Topics JSON	✓
3	HybridChunker	Segmenta documento en chunks semánticos estables	DocumentIR+Topics	DocumentChunks JSON	✓

		— schemas.py	# Contratos Pydantic (DocumentTopics,
		— utils.py	# Normalización, stopwords, embedding.
		— hybrid/	# Núcleo del modo híbrido
		— hybrid_contextizer.py	# Fusión TF-IDF + KeyBERT + embedding.
		— density_clustering.py	# Clustering semántico adaptativo
		— keyword_fusion.py	# Fusión híbrida de scores
		— mmr.py	# Maximal Marginal Relevance (diversi
		— analyzers.py	# Analizadores estadísticos por bloques
		— metrics_ext.py	# Métricas de topic-coherence y redundancia
		— __init__.py	
		— __init__.py	
		— schema_selector/	# Adaptive Schema Selector
		— registry_embeddings.py	# Genera Embeddings desde Registry
		— registry.py	# Ontologías y dominios (medical, legal, etc.)
		— schemas.py	# Contratos Pydantic
		— selector.py	# Lógica de scoring y selección adaptativa
		— utils.py	# Funciones auxiliares (similitud, normalización)
		— __init__.py	
		— mentions/	
		— llm_client.py	# Conexión a los modelos LLM
		— llm_extractor.py	# Maestro para extraer la Entidades
		— metrics.py	# Métricas de desempeño del Modelo
		— schemas.py	# Contratos Pydantic
		— prompt_builder.py	# Creador de Prompt aumentativo
		— utils.py	# Funciones auxiliares (similitud, normalización)
		— __init__.py	
		— pipelines/	# Configuración declarativa del pipeline
		— pipeline.yaml	
		— outputs_ir/	# Salidas intermedias (IR JSON)
		— outputs_doc_topics/	# DocumentIR + Topics (doc-level)
		— outputs_chunks/	# Chunks enriquecidos (chunk-level)
		— outputs_schema/	# Selección de esquemas adaptativos
		— neo4j/	
		— conf/	
		— neo4j.conf	
		— data/	
		— logs/	
		— import/	
		— plugins/	
		— .env	
		— docker-compose.neo4j.yml	
		— t2g_cli.py	# CLI unificado para orquestación
		— requirements.txt	
		— README.md	

Etapas explicadas

1) Parser (Doc → IR)

Entrada: PDF / DOCX / PNG / JPG

Salida: DocumentIR (outputs_ir/{DOC}_*.json)

Qué hace:

- Detecta formato y selecciona parser:
 - **PDF:** texto + tablas (pdfplumber); OCR fallback si escaneado.
 - **DOCX:** párrafos, headings, tablas (python-docx).
 - **IMG:** OCR (pytesseract).
- Normaliza espacios, guiones cortados, saltos de línea.
- Añade metadatos: sha256, mime, page_count, size_bytes.

Ejemplo de salida:

```
{
  "doc_id": "DOC-12345",
  "pages": [
    {
      "page_number": 1,
      "blocks": [
        {"type": "paragraph", "text": "Los leones viven en África..."},
        {"type": "table", "cells": [{"row": 0, "col": 0, "text": "Dato"}]}
      ]
    }
  ],
  "meta": {
    "mime": "application/pdf",
    "page_count": 1,
    "sha256": "...",
    "source_path": "docs/leones.pdf"
  }
}
```

2) Hybrid Contextizer (doc-level)

Entrada: DocumentIR (outputs_ir/*.json)

Salida: DocumentIR+Topics (outputs_doc_topics/*.json)

Qué hace (paso a paso):

1. Extracción y normalización del texto

- Toma todos los bloques textuales (`pages[] . blocks[] . text`) del IR.
- Aplica limpieza ligera:
 - Colapsa espacios en blanco.
 - Elimina caracteres no textuales (`•`, `¶`, `—`, etc.).
 - Sustituye comillas y apóstrofes para uniformidad.
- Cada bloque se conserva con su trazabilidad:
 - `page_number`, `block_index`, `type`.
- Si el texto está vacío o contiene menos de 3 caracteres, se descarta.

Resultado: un conjunto de bloques válidos `texts` de tamaño n , normalizados y listos para análisis semántico.

2. Cálculo de embeddings globales

Se usa el modelo configurado (`SentenceTransformer` con `cfg.embedding_model`).

$$E_i = f_S T(b_i)$$

donde cada E_i es un vector en $\mathbb{R}^d$.

Luego se calcula un vector promedio para representar el contexto general del documento:

$$\bar{E}_{doc} = (1/n) \times \sum E_i$$

Este embedding global sirve para medir coherencia temática y variación semántica entre bloques.

3. Router adaptativo (heurísticas del modo híbrido)

El módulo `analyzer.py` determina si debe ejecutarse el **modo híbrido**.

Evalúa tres métricas simples pero efectivas:

Métrica	Fórmula	Umbral	Significado
Longitud media (<code>avg_len</code>)	$\frac{1}{n} \sum b_i $	< 45	texto corto o ruidoso
Diversidad léxica (TTR)	$\frac{V}{T}$	> 0.5	mucha variación léxica
Varianza semántica (<code>semantic_var</code>)	$\text{Var}(E_i)$	> 0.25	temas dispersos

El híbrido se activa si **2 o más** condiciones son verdaderas:

```
"reason": "doc-hybrid"
```

Este paso evita usar métodos costosos de clustering o reducción de dimensión en textos pequeños o de baja densidad.

4. Clustering semántico por densidad

Se aplica **DBSCAN** directamente sobre los embeddings para detectar grupos semánticos sin predefinir `n_topics`:

$$cluster(E_i) = k, sidist_{cosine}(E_i, E_j) < \varepsilon - 1, \text{ si } i \neq j$$

Parámetros:

$$\varepsilon = 0.25; min_samples = 2$$

La ventaja de DBSCAN es que **no requiere conocer cuántos temas existen**; se adapta a la estructura semántica del documento.

5. Construcción de tópicos y keywords

Una vez formados los clusters, el módulo `density_clustering.py` construye tópicos equivalentes a `TopicItem`:

$$topics = \{t_k = (keywords, exemplar, count)\}$$

Cada tópico t_k se resume con:

- **Exemplar:** bloque más representativo (máxima similitud media dentro del cluster).
- **Count:** número de bloques asignados.
- **Keywords:** extraídas mediante la fusión híbrida de señales léxicas y semánticas.

6. Extracción y fusión de keywords (TF-IDF + KeyBERT + Embeddings)

Se combina información de tres fuentes:

1. Relevancia léxica (TF-IDF):

$$S_{tfidf}(w) = tf(w) \cdot \log \frac{N}{df(w)}$$

Evalúa la importancia del término dentro del documento.

2. Relevancia contextual (KeyBERT, opcional):

$$S_{keybert}(w) = \cos(\vec{w}, \vec{E}_{doc})$$

Mide alineación semántica con el contexto global.

3. Cohesión semántica (embeddings):

$$S_{hybrid}(w) = 0.5 \cdot S_{tfidf}(w) + 0.3 \cdot S_{keybert}(w) + 0.2 \cdot S_{emb}(w)$$

Las tres se fusionan ponderadamente:

$$S_{hybrid}(w) = 0.5 S_{tfidf}(w) + 0.3 S_{keybert}(w) + 0.2 S_{emb}(w)$$

Esta ponderación surge de experimentos que equilibran precisión contextual y estabilidad en documentos pequeños.

7. Selección de keywords por Maximal Marginal Relevance (MMR)

Se aplica el filtro MMR (`mmr.py`) para eliminar sinónimos y redundancia:

$$\text{MMR}(w_i) = \lambda \cos(\vec{w}_i, \vec{t}) - (1 - \lambda) \max_{w_j \in S} \cos(\vec{w}_i, \vec{w}_j)$$

con $\lambda = 0.7$.

Resultado: un conjunto reducido de keywords informativas y no redundantes.

8. Cálculo de métricas extendidas

Las métricas cuantitativas (metrics_ext.py) permiten auditar la calidad semántica:

Métrica	Descripción	Fórmula
entropy_topics	Dispersión de tópicos	$-\sum p_j \log p_j$
redundancy_score	Redundancia media	$1 - \frac{V_{\text{único}}}{V}$
keywords_diversity_ext	Diversidad global	$\frac{ V_{\text{único}} }{ V }$
semantic_variance	Varianza de embeddings	$\text{Var}(E_{\text{exemplar}})$
coherence_semantic	Coherencia intra-tópico	$\cos(E_{kw_i}, E_{kw_j})$

9. Salida (JSON)

```
{
  "meta": {
    "topics_doc": {
      "reason": "doc-hybrid",
      "n_samples": 25,
      "n_topics": 3,
      "keywords_global": ["zacks", "research", "southern", "industry", "ra
      "topics": [
        {"topic_id": 0, "count": 3, "keywords": ["zacks", "research", "equ
        {"topic_id": 1, "count": 2, "keywords": ["+1.00%", "+0.36%"]},
        {"topic_id": 2, "count": 2, "keywords": ["rank", "strong", "hold"]}
      ],
      "metrics_ext": {
        "entropy_topics": 0.982,
        "semantic_variance": 0.425,
        "coherence_semantic": 0.303,
        "keywords_diversity_ext": 0.958,
        "topic_balance": 0.798,
        "informativeness_ratio": 1.0,
        "redundancy_score": 0.0
      }
    }
  }
}
```

10. Beneficios técnicos del enfoque híbrido

Dimensión	Mejora
Robustez	Maneja textos breves y ruidosos sin colapsar.
Interpretabilidad	Cada tópico conserva su contexto original.
Estabilidad	No requiere hiperparámetros ajustados.
Trazabilidad	Cada decisión se justifica con <code>meta.reason</code> y métricas.
Escalabilidad	Reutiliza embeddings y evita pasos costosos.

3) HybridChunker

Entrada: DocumentIR+Topics (outputs_doc_topics/*.json)

Salida: DocumentChunks (outputs_chunks/*.json)

Qué hace (paso a paso):

1. Extracción de bloques base

- Toma todos los bloques textuales del DocumentIR.
- Cada bloque conserva trazabilidad (`page_number`, `block_indices`) para poder mapear chunks a posiciones exactas en el documento.
- Filtra bloques vacíos o no textuales (imágenes, tablas sin OCR).

2. Segmentación semántica híbrida

Se combinan varias estrategias para dividir el documento en **chunks coherentes de ≤ 2048 tokens** (óptimo para LLMs):

- **Reglas de headings:** patrones típicos (Introducción, Métodos, Conclusiones, etc.) detectados vía regex.
Esto evita cortar secciones temáticas de forma arbitraria.
- **spaCy sentence boundaries:** segmenta párrafos largos en oraciones completas, preservando la coherencia sintáctica.
Si spaCy no está disponible, se aplica un fallback mediante puntuación (`.`, `,`, `;`, `?`, `!`) o saltos de línea dobles (`\n\n`).
- **Empaquetado semántico por longitud:** agrupa oraciones hasta alcanzar un límite aproximado de tokens.
Controla umbrales de tamaño (`min_chars`, `max_chars`, `max_tokens`) para mantener chunks **equilibrados** en densidad y contexto.
→ El resultado son **unidades estables**: suficientemente largas para el contexto, pero sin sobrepasar los límites óptimos de procesamiento.

3. Herencia de contexto (`topic_hints`)

- Cada chunk hereda información del doc-level contextizer (`topic_ids`, `keywords_global`, `topic_affinity`).
- Esto asegura **consistencia semántica vertical**: lo que el documento conoce a nivel global se transfiere a los fragmentos locales.
- La afinidad entre el chunk y los tópicos globales se calcula mediante una mezcla semántico-léxica:

$$\text{topic affinity blend}(c, t) = \alpha \cdot \cos(\vec{c}, \vec{t}) + (1 - \alpha) \cdot J(c, t)$$

Donde:

- $\vec{c}$ y $\vec{t}$ son los embeddings del chunk y del topic.
- $J(c, t)$ es la similitud léxica (*Jaccard*).
- α controla el peso semántico (por defecto, $\alpha = 0.7$).

Esta combinación hace al sistema más robusto frente a textos cortos (tweets, cláusulas legales o notas clínicas) donde la semántica sola puede ser insuficiente.

4. Cálculo de métricas locales

Para evaluar la calidad y la coherencia de los chunks, se calculan métricas cuantitativas:

- `cohesion_vs_doc`: mide la similitud coseno entre el embedding del chunk y el embedding promedio del documento.
Representa **qué tan bien el fragmento conserva el contexto global**.

$$\text{cohesion vs doc}(c_i) = \cos(\vec{c}_i, \vec{D})$$

- `max_redundancy`: mide la similitud máxima del chunk con cualquier otro chunk dentro del mismo documento.
Detecta **fragmentos repetitivos o duplicados**.

$$\text{max redundancy}(c_i) = \max_{j \neq i} \cos(\vec{c}_i, \vec{c}_j)$$

- `redundancy_norm`: versión normalizada de la redundancia, que ajusta el valor según el tamaño relativo del fragmento.

$$\text{redundancy norm}(c_i) = \text{max redundancy}(c_i) \times \frac{\text{len}(c_i)}{\text{avg len}(\text{chunks})}$$

Esto penaliza más a los chunks **largos y redundantes**, y reduce el impacto de los fragmentos **cortos pero similares**.

En textos como contratos, reseñas o tweets, mejora la detección de contenido **repetitivo** versus **informativo**.

- `novelty`: mide la proporción de información nueva que aporta cada fragmento.
Es complementaria a la redundancia.

$$\text{novelty}(c_i) = 1 - \text{max redundancy}(c_i)$$

Un valor alto de `novelty` indica que el chunk aporta **contexto único o evidencia nueva**.

- `chunk_health`: métrica compuesta que pondera la cohesión y la novedad penalizando la redundancia.

Resume la **salud semántica del fragmento**.

$$\text{chunk_health}(c_i) = \text{cohesion vs doc}(c_i) \times (1 - \text{max redundancy}(c_i))$$

Este score puede usarse en etapas posteriores (por ejemplo, el **Adaptive Schema Selector**) para **ponderar o filtrar chunks** según su calidad semántica.

5. Serialización robusta

- Cada chunk se guarda con un `chunk_id` único, metadatos (`doc_id`, idioma, embeddings opcionales) y trazabilidad (`source_spans`).
- El formato JSON conserva compatibilidad con las etapas siguientes (`contextize-chunks`, `schema-select`).
- Si los embeddings o spaCy no están disponibles, aplica **fallbacks automáticos** para mantener la robustez del pipeline.

Ejemplo de salida (simplificado):

```
{
  "chunk_id": "DOC-123_0001",
  "text": "Complicaciones Asociadas al tratamiento prolongado...",
  "topic_hints": {
    "inherited_keywords": ["diabetes", "tratamiento"],
    "inherited_topic_ids": [0, 1],
    "topic_affinity_blend": {"0": 0.82, "1": 0.71}
  },
  "scores": {
    "cohesion_vs_doc": 0.82,
    "max_redundancy": 0.59,
    "redundancy_norm": 0.73,
    "novelty": 0.41,
    "chunk_health": 0.34
  },
  "meta_local": {
    "embedding_model": "all-MiniLM-L6-v2",
    "lang": "es"
  }
}
```

4) Hybrid Contextizer (chunk-level)

Entrada: DocumentChunks (outputs_chunks/*.json)

Salida: Chunks+Topics (outputs_chunks/*.json)

Qué hace (paso a paso):

1. Carga de chunks y herencia vertical

- Toma los chunks creados por el HybridChunker.
- Cada chunk incluye texto, contexto heredado (topic_hints, keywords_global) y métricas locales (cohesion_vs_doc, chunk_health).
- Se asegura **consistencia semántica vertical**: Cada chunk llega al **Contextizer (chunk-level)** con su contexto semántico ya heredado desde el documento, a través de los campos topic_hints, keywords_global y métricas locales (cohesion_vs_doc, chunk_health). En la versión actual del pipeline, esta etapa opera **por defecto en modo LIGHT**, lo que significa que:
 - No ejecuta clustering adicional ni reducción de dimensión.
 - Reutiliza los topic_hints como **priors semánticos**.
 - Refina keywords locales mediante una fusión híbrida (TF-IDF, KeyBERT y embeddings).

Esto garantiza una **consistencia semántica vertical automática** entre documento y chunks, manteniendo alineación de tópicos globales y subtemas locales sin duplicar cómputo ni perder coherencia.

$T_{\text{chunk}} \subseteq T_{\text{doc}}$

Esto garantiza que los subtemas locales siempre estén dentro de los temas globales del documento.

2. Recálculo de embeddings locales

Cada chunk c_i se representa con un vector SentenceTransformer:

$E_{c_i} = f_{ST}(c_i)$

Y se calcula un embedding global del documento:

$\bar{E}_{\text{doc}} = \frac{1}{N} \sum_i E_{c_i}$

Este vector sirve para medir la alineación temática entre cada fragmento y el contexto general.

3. Router adaptativo (modo de operación)

Condición	Modo	Acción
n_samples = 0	skip	omite
< 5	fallback-small	usa TF-IDF simple
5 ≤ n ≤ 50	hybrid	usa pipeline híbrido completo
> 50	hybrid-large	usa DBSCAN más estricto

```
"reason": "chunk-hybrid"
```

4. Generación de tópicos locales

- **TF-IDF fallback:**

$$S_{\text{freq}}(w) = \frac{f(w)}{\sum f(w)}$$

- **Modo híbrido completo:**

$$S_{\text{hybrid}}(w) = 0.5S_{\text{tfidf}}(w) + 0.3S_{\text{keybert}}(w) + 0.2S_{\text{emb}}(w)$$

- **Clustering local:**

$$\varepsilon = 0.25, \quad \text{min_samples} = 2$$

Cada cluster produce un conjunto de keywords diversificado con MMR local.

5. Afinidad entre tópicos locales y globales

Se mide la coherencia semántica entre cada chunk y los tópicos globales del documento:

$$\text{affinity}(c_i, t_j) = 0.7 \cos(E_{c_i}, E_{t_j}) + 0.3J(c_i, t_j)$$

donde J es similitud léxica (*Jaccard*).

6. Métricas intra-chunk

Métrica	Descripción	Fórmula
local_cohesion	coherencia con el cluster	$\cos(E_{c_i}, \bar{E}_{\text{cluster}})$
local_redundancy	similitud con chunks vecinos	$\max_{j \neq i} \cos(E_{c_i}, E_{c_j})$
novelty	información nueva	$1 - \text{local_redundancy}$
chunk_health	balance de calidad	$\text{local_cohesion} \times (1 - \text{local_redundancy})$

Estas métricas permiten identificar fragmentos repetitivos o irrelevantes.

7. Salida (JSON)

```
{  
  "chunks": [  
    {
```

```
"chunk_id": "DOC-A5E2_0000_895D9A2D78",
"text": "Zacks Equity Research Mon, October 6, 2025 ...",
"topic": {
  "topic_id": 0,
  "keywords": ["zacks", "research", "southern", "industry"],
  "prob": 1.0
},
"topic_hints": {
  "inherited_keywords": ["zacks", "southern", "industry"],
  "inherited_topic_ids": [0, 1, 2],
  "topic_affinity": {"0": 0.25, "1": 0.18, "2": 0.26}
},
"scores": {
  "cohesion_vs_doc": 1.0,
  "chunk_health": 1.0,
  "lexical_density": 1.0,
  "type_token_ratio": 0.48
}
},
],
"meta": {
  "topics_chunks": {
    "reason": "chunk-hybrid",
    "n_topics": 1,
    "metrics_ext": {
      "context_alignment": 0.0,
      "redundancy_penalty": 0.0
    }
  }
}
}
```

8. Beneficios técnicos (chunk-level)

Dimensión	Mejora
Granularidad	Detecta subtemas dentro de secciones extensas.
Consistencia vertical	Mantiene alineación semántica con tópicos globales.
Resiliencia	Funciona incluso con pocos chunks o texto corto.
Métricas internas	Evalúa coherencia, redundancia y novedad.
Escalabilidad	Procesa grandes volúmenes en paralelo.
Reutilización	Aprovecha embeddings previos del doc-level.

5) Adaptive Schema Selector

Entrada: Chunks+Topics (outputs_chunks/*.json)
Salida: SchemaSelection (outputs_schema/*.json)

Qué hace:

El **Adaptive Schema Selector (ASS)** determina dinámicamente qué **dominios de entidades** (por ejemplo, médico, legal, financiero o genérico) son relevantes para cada documento y chunk. Su propósito es **filtrar, priorizar y contextualizar** los tipos de entidades que deben extraerse en las etapas siguientes, mejorando la **precisión semántica** del grafo y reduciendo ruido.

1. Registro de dominios (registry.py)

Cada dominio está definido en la ontología base (registry.py) y contiene:

- **Entidades (EntityTypeDef)** con atributos (por ejemplo: Disease, Treatment, Contract, Transaction).
- **Relaciones (RelationTypeDef)** entre entidades (por ejemplo: treated_with, paid_by, binds).
- **Aliases** y vocabulario específico en español e inglés (por ejemplo: "enfermedad", "patología", "disease" para Disease).
- **Descripciones semánticas** utilizadas para generar embeddings de referencia.

Los dominios incluidos en la versión v2_bilingual son:

Dominio	Ejemplo de entidades	Contextos típicos
medical	Disease, Symptom, Drug, Treatment, LabTest	artículos clínicos, diagnósticos
legal	Contract, Party, Obligation, Penalty	contratos, cláusulas, litigios
financial	Invoice, Transaction, StockIndicator, Policy	facturas, informes financieros
reviews_and_news	Review, NewsArticle, MarketEvent	reseñas, noticias económicas
ecommerce	Order, Product, Review	comercio electrónico, reseñas de clientes
identity	Person, Address, IDDocument	registros, formularios
generic	Person, Organization, Date, Location, Amount	fallback universal

2. Extracción de señales del documento/chunk

El selector combina **tres tipos de señales** para estimar la afinidad de cada texto con los dominios registrados:

- **Keywords**

Se detectan coincidencias entre los tokens normalizados del documento y los **alias** del dominio.

Las coincidencias se ponderan por frecuencia y relevancia POS (sustantivos, nombres propios, etc.).

$$S_{\text{kw}}(d) = \frac{\text{overlaps}(d)}{\text{total aliases}(d)} \times \log(1 + f_{\text{term}})$$

Donde:

- $\text{overlaps}(d)$ → número de alias del dominio encontrados.
- f_{term} → frecuencia media de los términos coincidentes.

Ejemplo: un documento con "contrato", "firma", "cláusula" activará el dominio `legal` con alto S_{kw} .

- **Embeddings**

Calcula la similitud coseno entre los **embeddings promedio del texto** y los **embeddings representativos del dominio** (precalculados a partir de sus descripciones y alias).

$$S_{\text{emb}}(d) = \cos(\vec{v}_{\text{text}}, \vec{v}_{\text{domain}})$$

- $\vec{v}_{\text{text}}$ → embedding medio del chunk o documento.
- $\vec{v}_{\text{domain}}$ → embedding medio del dominio.

Ejemplo: "antihipertensivo" activa el dominio `medical` aunque la palabra "enfermedad" no aparezca explícitamente.

- **Priors**

Cada dominio tiene un peso base $P(d)$ que refleja su probabilidad a priori de aparecer.

$$P(d) = \text{prior}(d) \in [0, 1]$$

Ejemplo: `generic` = 0.1, `medical` = 0.05, `legal` = 0.05
El dominio `generic` siempre se considera como fallback.

3. Fórmula de scoring (por dominio)

Para cada dominio d , se calcula un score ponderado combinando las tres señales:

$$\text{score}(d) = \alpha \cdot S_{\text{kw}}(d) + \beta \cdot S_{\text{emb}}(d) + \gamma \cdot P(d)$$

Donde:

- $S_{\text{kw}}(d)$: score normalizado por coincidencia léxica.
- $S_{\text{emb}}(d)$: similitud coseno entre embeddings.
- $P(d)$: prior asignado al dominio.
- α, β, γ : hiperparámetros configurables en `SelectorConfig`.

Ejemplo default:

$$\alpha = 0.6, \quad \beta = 0.3, \quad \gamma = 0.1$$

→ más peso a keywords, menor a embeddings y priors.

En textos técnicos (contratos, facturas) domina α .

En textos conceptuales (reseñas o informes), β captura mejor la afinidad semántica.

4. Selección final

Una vez calculados los scores, se aplica la fase de decisión:

- Se **ordenan** los dominios de mayor a menor score.
- Se **descartan** los dominios con $\text{score} < \text{min_topic_conf}$.
- Se **seleccionan** los top-k dominios configurados (por defecto $\text{topk_domains} = 2$).
- Se marca $\text{ambiguous}=\text{True}$ si la diferencia entre el primer y segundo dominio es menor al margen definido:

$$\text{ambiguous} = |S(d_1) - S(d_2)| < \tau$$

donde τ es $\text{ambiguity_threshold}$ (por defecto 0.1).

- Siempre se incluye el dominio **genérico** como fallback ($\text{allow_fallback_generic}=\text{True}$).

Ejemplo de salida (simplificado):

```
{
  "doc": {
    "doc_id": "DOC-CA28DAF58CC7",
    "top_domains": ["financial", "reviews_and_news"],
    "ambiguous": false,
    "domain_scores": [
      {
        "domain": "financial",
        "score": 0.27,
        "evidence": [
          {"kind": "keyword", "detail": {"match": ["pago", "banco", "transac"], "value": 0.84}},
          {"kind": "embedding", "detail": {"cosine": 0.84}},
          {"kind": "prior", "detail": {"value": 0.05}}
        ]
      },
      {
        "domain": "reviews_and_news",
        "score": 0.15,
        "evidence": [
          {"kind": "keyword", "detail": {"match": ["noticia", "acciones"], "value": 0.72}},
          {"kind": "embedding", "detail": {"cosine": 0.72}}
        ]
      }
    ]
  }
}
```



```

    ]
  },
  "chunks": [
    {
      "chunk_id": "DOC-CA28DAF58CC7_0001",
      "top_domain": "financial",
      "ambiguous": false,
      "domain_scores": [
        {"domain": "financial", "score": 0.31},
        {"domain": "reviews_and_news", "score": 0.12}
      ]
    }
  ],
  "meta": {
    "alpha": 0.6,
    "beta": 0.3,
    "gamma": 0.1,
    "always_include": ["generic"],
    "ambiguity_threshold": 0.1,
    "registry_version": "v2_bilingual"
  }
}

```

Notas adicionales:

- La afinidad entre dominios puede ajustarse con un refuerzo contextual:

$$S_{\text{adj}}(d, c_i) = S_{\text{domain}}(d, c_i) \times (1 + \lambda \cdot \text{cohesion_vs_doc}(c_i))$$

donde $\lambda = 0.2$ pondera la cohesión semántica entre chunk y documento.

- La inferencia final combina contexto global y local:

$$S_{\text{final}}(d) = \omega \cdot S_{\text{doc}}(d) + (1 - \omega) \cdot \frac{1}{N} \sum_{i=1}^N S_{\text{chunk}}(d, c_i)$$

con $\omega = 0.5$ por defecto.

Beneficios:

- **Precisión contextual:** sólo se procesan entidades coherentes con el dominio dominante.
 - **Escalabilidad:** nuevos dominios pueden añadirse fácilmente al `registry.py`.
 - **Interpretabilidad:** cada decisión conserva su `evidence_trace` (palabras clave, similitudes, priors).
 - **Auditable:** todos los pesos, fórmulas y umbrales se guardan en el meta del JSON.
 - **Consistencia vertical:** mantiene coherencia entre documento y chunks.
-

5.1 Generación y Consumo de Embeddings de Dominios (`registry_embeddings.py`)

El módulo `registry_embeddings.py` genera los **vectores semánticos representativos** de cada dominio definido en la ontología (`registry.py`). Estos embeddings sirven como **referencia base** para calcular la similitud coseno entre el texto (documento o chunk) y los dominios disponibles durante la selección adaptativa.

1. Ejecución del generador

El generador puede ejecutarse de dos maneras equivalentes:

```
# Opción 1 – Modo directo
python schema_selector/registry_embeddings.py

# Opción 2 – Modo paquete (recomendada)
python -m schema_selector.registry_embeddings
```

En ambos casos, el script:

1. Carga los dominios y sus alias desde `registry.py`.
2. Usa el modelo `all-MiniLM-L6-v2` (el mismo del Contextizer) para obtener embeddings por alias.
3. Calcula el **centroide normalizado** de cada dominio (promedio vectorial de sus alias).
4. Guarda los resultados en un archivo de cache local:

```
schema_selector/registry_vectors.json
```

2. Ejemplo de salida

```
Embeddings cargados desde cache (9 dominios, 0 faltantes).
Archivo: /Users/martinjurado/Desktop/prjs/T2G/schema_selector/registry_vec
Recalculando embeddings de dominios...
Embeddings de dominios guardados en /Users/martinjurado/Desktop/prjs/T2G/s
Validación: 9/9 dominios con embeddings (384 dim).
```

Cada dominio queda asociado a un vector centroid de 384 dimensiones, por ejemplo:

```
{
  "medical": {
    "centroid": [0.123, -0.045, 0.078, ...]
  },
  "legal": {
    "centroid": [0.102, -0.030, 0.091, ...]
  }
}
```

3. Relación con el Adaptive Schema Selector

Durante la etapa de selección (`selector.py`), estos embeddings son **automáticamente cargados** al inicializar el módulo. Cada dominio del registro (`OntologyDomain`) posee ahora un atributo adicional:

```
domain.label_vecs["centroid"]
```

El selector utiliza estos vectores para calcular el componente de **similitud semántica**:

$$S_{emb}(d) = \cos(v_{text}, v_{domain})$$

donde:

- `v_text` → embedding promedio del documento o chunk, obtenido con el mismo modelo del **Contextizer**.
- `v_domain` → centroide precomputado y persistido en `registry_vectors.json`.

4. Sincronización y regeneración

Cuando se agregan nuevos dominios o alias en `registry.py`, es necesario **recalcular los embeddings** para mantener la coherencia semántica:

```
python -m schema_selector.registry_embeddings
```

Esto sobrescribe el archivo `registry_vectors.json` con los nuevos centroides, sin afectar al registro base.

5. Integración con el flujo del pipeline

El proceso completo puede representarse así:

```
registry.py → registry_embeddings.py → registry_vectors.json → selector.py
```

1. `registry.py` define dominios, entidades, relaciones y alias.
2. `registry_embeddings.py` transforma esos alias en embeddings promedio (centroides).
3. `registry_vectors.json` almacena los vectores de referencia por dominio.
4. `selector.py` carga y compara estos vectores contra los embeddings de cada documento/chunk.
5. El resultado se refleja en `outputs_schema/*.json`, con los scores de afinidad por dominio.

Beneficio principal: La incorporación del archivo `registry_vectors.json` permite **medir afinidad semántica real**, no solo coincidencia léxica, habilitando detección de contexto incluso

cuando no aparecen términos literales del dominio (por ejemplo, "cláusula de confidencialidad" activa legal aunque no diga "contrato").

6) Mentions (NER/RE) ➡️

- Detecta menciones de entidades **condicionadas por tópicos**.

7–11) Clustering → Label → LLM → Normalización → Grafo ➡️

- Agrupan spans, etiquetan con weak supervision, refinan con LLM y normalizan entidades.
- Export final a **grafo de conocimiento**.

 Flujo de información en T2G (Contrato de datos)

Etapa / Subsistema	Entrada (qué toma)	Salida (qué agrega / construye)
1. Parser (Doc → IR)	Documento bruto (PDF, DOCX, IMG)	DocumentIR: texto por bloques, tablas, metadatos (mime, pages, sha256, source_path).
2. Contextizer (doc-level)	DocumentIR.pages.blocks.text	meta.topics_doc: tópicos globales, keywords generales, ejemplares, outlier_ratio.
3. HybridChunker	DocumentIR+Topics	chunks[*]: segmentos ≤2048 tokens. Heredan topic_hints + métricas (cohesion, redundancy).
4. Contextizer (chunk-level)	chunks.text + topic_hints heredados	chunks[*].topic: tópico local con topic_id, keywords, prob + meta.topics_chunks.
5. Schema Selector (adaptativo)	chunks+topics + registry	SchemaSelection: dominios relevantes por doc y chunk + evidence_trace (keywords, embeddings, priors, scores).

 Pipeline declarativo (YAML)

Archivo: pipelines/pipeline.yaml

```
pipeline:
  dry_run: false
  continue_on_error: true

stages:
  - name: parse
```

```
args:
  clean_outdir: true
  inputs_glob:
    - "docs/*.pdf"
    - "docs/*.png"
    - "docs/*.jpg"
    - "docs/*.docx"
  outdir: "outputs_ir"

- name: contextize-doc
  args:
    clean_outdir: true
    ir_glob: "outputs_ir/*.json"
    embedding_model: "all-MiniLM-L6-v2"
    nr_topics: null
    seed: 42
    outdir: "outputs_doc_topics"

- name: chunk
  args:
    clean_outdir: true
    ir_glob: "outputs_doc_topics/*.json"
    embedding_model: "all-MiniLM-L6-v2"
    spacy_model: "es_core_news_sm"
    max_tokens: 2048
    min_chars: 280
    outdir: "outputs_chunks"

- name: contextize-chunks
  args:
    clean_outdir: false
    ir_glob: "outputs_chunks/*.json"
    embedding_model: "all-MiniLM-L6-v2"
    nr_topics: null
    seed: 42
    outdir: "outputs_chunks"

- name: schema-select
  args:
    chunks_glob: "outputs_chunks/*.json"
    outdir: "outputs_schema"
    clean_outdir: true
    alpha_kw: 0.6
    beta_emb: 0.3
    gamma_prior: 0.1
    ambig_margin: 0.08
```

```
topk_domains: 2
topk_entity_types: 5
```

Ejecutar:

```
python t2g_cli.py pipeline-yaml
```

Métricas por subsistema

Parser

Evalúa la calidad y consistencia del parseo de documentos heterogéneos.

- `percent_docs_ok`: proporción de documentos parseados sin errores.
- `layout_loss`: pérdida de estructura visual o de formato.
- `table_consistency`: coherencia entre tablas detectadas y esperadas.
- `ocr_ratio`: porcentaje de páginas procesadas mediante OCR (indicador de calidad visual).
- `avg_parse_time`: tiempo promedio de procesamiento por documento.
- `block_density`: número promedio de bloques válidos por página.

Contextizer (doc-level)

Métricas principales:

- `coverage`: proporción de bloques asignados a algún tópico.
- `outlier_rate`: ratio de bloques descartados por ruido o baja densidad.
- `topic_size_stats`: distribución de tamaño de clusters (`min`, `median`, `p95`).
- `keywords_diversity`: diversidad de palabras clave únicas (riqueza semántica).
- `topic_stability`: correlación promedio entre embeddings de tópicos en ejecuciones sucesivas.
- `topic_entropy`: medida de dispersión temática (mayor = más heterogeneidad).

Métricas extendidas (doc-level):

- `entropy_topics`: dispersión de masa entre tópicos (equilibrio global).
- `redundancy_score`: repetición media entre keywords de un mismo tema (menor = mejor).
- `keywords_diversity_ext`: diversidad ampliada considerando keywords post-MMR.
- `semantic_variance`: varianza entre embeddings de *exemplars* (cohesión semántica global).
- `coherence_semantic`: similitud promedio entre keywords dentro de cada tópico.
- `context_quality`: índice compuesto de calidad del modelado temático.

HybridChunker

Evalúa la **coherencia**, **redundancia** y **salud semántica** de los fragmentos.

◆ Métricas base

- `chunk_length_stats`: distribución de tamaños (caracteres / tokens).
- `cohesion_vs_doc`: similitud coseno entre embedding de chunk y embedding global del documento.

$$\text{cohesion_vs_doc}(c_i) = \cos(\vec{c}_i, \vec{D})$$

- `max_redundancy`: similitud máxima entre embeddings de chunks.

$$\text{max redundancy}(c_i) = \max_{j \neq i} \cos(\vec{c}_i, \vec{c}_j)$$

- `redundancy_norm`: redundancia ajustada por longitud.

$$\text{redundancy norm}(c_i) = \text{max redundancy}(c_i) \times \frac{\text{len}(c_i)}{\text{avg len}(\text{chunks})}$$

- `novelty`: proporción de información nueva aportada.

$$\text{novelty}(c_i) = 1 - \text{max redundancy}(c_i)$$

◆ Métricas compuestas

- `chunk_health`: salud semántica = cohesión × (1 – redundancia).
- `semantic_density`: proporción de tokens relevantes (sin stopwords) sobre el total.
- `lexical_density`: densidad léxica medida por términos significativos / totales.
- `type_token_ratio`: diversidad de vocabulario (variedad léxica).
- `semantic_coverage`: % de chunks con cohesión ≥ 0.7 (bien alineados al documento).
- `redundancy_flag_rate`: % de chunks con redundancia excesiva ≥ 0.6 .
- `topic_affinity_blend`: afinidad semántico-léxica con los tópicos globales del documento.

$$\text{topic affinity blend}(c, t) = \alpha \cos(\vec{c}, \vec{t}) + (1 - \alpha) J(c, t)$$

◆ Métricas globales

- `global_health_score`: indicador compuesto (good, moderate, poor).
- `avg_chunk_health`: promedio de salud semántica global.
- `coverage_ratio`: proporción de texto total cubierto por chunks válidos.
- `oversegmentation_rate`: % de chunks demasiado pequeños (bajo umbral `min_chars`).
- `undersegmentation_rate`: % de chunks demasiado largos (superan `max_tokens`).

Contextizer (chunk-level)

Evalúa la coherencia temática local y su relación con los tópicos globales.

Métricas principales:

- `coverage`: % de chunks con tópico asignado.
- `fallback_rate`: % de documentos donde se usó fallback en lugar de clustering.
- `topic_size_stats`: distribución de tamaños de subtemas.
- `keywords_overlap`: solapamiento promedio entre keywords globales y locales.
- `topic_coherence_local`: coherencia intracluster promedio (similitud coseno media).
- `local_entropy`: dispersión de tópicos locales (mide estabilidad semántica).

Métricas extendidas (chunk-level):

- `redundancy_penalty`: penalización por similitud excesiva entre embeddings de chunks.
- `context_alignment`: correlación promedio entre `topic_hints` heredados y los detectados localmente.

Adaptive Schema Selector

Evalúa la **relevancia y precisión contextual** del mapeo dominio–documento.

◆ Métricas base

- `domain_score_distribution`: histograma de scores por dominio.
$$\text{score}(d) = \alpha S_{\text{kw}}(d) + \beta S_{\text{emb}}(d) + \gamma P(d)$$
- `coverage_domains`: número promedio de dominios relevantes por documento.
- `ambiguity_rate`: % de documentos o chunks marcados como `ambiguous = True`.
- `domain_confidence_gap`: diferencia entre el primer y segundo dominio (medida de separabilidad).
- `prior_influence`: peso efectivo de los priors sobre el score final.
- `always_included_rate`: % de documentos donde el dominio genérico fue incluido por fallback.

◆ Métricas de refuerzo contextual

- `contextual_boost_effect`: variación media del score tras aplicar refuerzo semántico.

$$\Delta S = S_{\text{adj}} - S_{\text{domain}}$$

- `lambda_effectiveness`: sensibilidad del refuerzo de cohesión (variación promedio por unidad de λ).

$$\eta_{\lambda} = \frac{\Delta S}{\lambda}$$

◆ Métricas de calidad ontológica

- `schema_alignment`: similitud promedio entre entidades detectadas y entidades esperadas del dominio.
- `entity_type_coverage`: % de tipos de entidad del dominio detectados al menos una vez.

- `relation_type_coverage`: % de relaciones del dominio identificadas.
- `ontology_diversity`: número de dominios distintos presentes en el corpus.

◆ Métricas globales

- `domain_precision`: proporción de dominios correctamente asignados (vs. gold standard si existe).
- `domain_recall`: proporción de dominios relevantes detectados.
- `domain_f1`: media armónica entre precisión y recall (solo si hay ground truth disponible).
- `evidence_trace`: trazabilidad completa de evidencias (keywords, embeddings, priors, scores).

Instalación

Requisitos principales

```
python3 -m venv venv
source venv/bin/activate
pip install -r requirements.txt
```

OCR

```
brew install tesseract tesseract-lang # macOS
sudo apt install tesseract-ocr-spa # Ubuntu/Debian
```

NLP / embeddings

```
pip install spacy sentence-transformers
python -m spacy download es_core_news_sm
python -m spacy download en_core_web_sm
```

Opcionales

```
pip install torch joblib matplotlib wordcloud
```

Uso rápido (CLI)

```
# Parser → IR
python t2g_cli.py parse docs/ejemplo.pdf --outdir outputs_ir

# Contextizer (doc) → IR+Topics
python t2g_cli.py contextize-doc outputs_ir/*.json --outdir outputs_doc_to
```

```
# Pipeline completo  
python t2g_cli.py pipeline-yaml
```

Troubleshooting

- **JSONs vacíos:** borra outputs y reejecuta.
- **OCR falla:** instala Tesseract y revisa PATH.
- **spaCy no carga:** descarga modelos es_core_news_sm, en_core_web_sm.
- **Errores de import:** ejecuta siempre desde raíz del proyecto.